Régression linéaire multiple par descente de gradient

Prédire la réussite des étudiants

1. Contexte

Une école veut savoir si la note finale d'un étudiant dans une matière peut être prédite à partir de comportements mesurables en cours de semestre. On dispose de 24 étudiants avec 5 variables explicatives et une note finale sur 20.

Variables (features) :

- heures_etude_semaine : temps de travail personnel hebdomadaire (heures)
- taux_presence_pct : taux de présence en cours (%)
- note_controle_continu : moyenne du contrôle continu (/20)
- heures_sommeil_nuit : sommeil moyen par nuit (heures)
- moyenne_annee_precedente : moyenne générale de l'année précédente (/20)

Cible (target) : note_finale (/20). Une colonne binaire réussite (1 si ≥ 10, 0 sinon).

Aperçu du dataset (fichier CSV complet fourni séparément, 24 étudiants) :

ID	heures étude	présence %	CC /20	sommeil h	moy. année préc. /20	note finale	réussite
E01	12.5	42	11.1	8.4	4.4	8.8	0
E02	17.9	71	17.7	7.3	9.6	13.8	1
E05	6.0	78	3.1	8.7	18.5	5.5	0
E09	15.9	41	16.7	5.0	17.1	14.4	1
E15	10.1	90	14.2	6.8	15.5	9.8	0
…	…	…	…	…	…	…	…

Taux de réussite global : 37,5 %, un déséquilibre volontaire pour que le modèle ait un vrai travail à faire.

2. Objectifs pédagogiques

- Écrire une hypothèse linéaire multivariée h_θ(x)
- Formuler la fonction de coût (erreur quadratique moyenne)
- Dériver à la main le gradient par rapport à chaque paramètre
- Implémenter la descente de gradient sans scikit-learn (numpy uniquement)
- Observer l'effet du taux d'apprentissage et de la normalisation des features

3. Partie 1, Modélisation

Avec x = (x1, x2, x3, x4, x5) les 5 variables et θ = (θ0, θ1, ..., θ5) les paramètres (θ0 = biais) :

$$h_\theta(x) = \theta 0 + \theta 1 \cdot x1 + \theta 2 \cdot x2 + \theta 3 \cdot x3 + \theta 4 \cdot x4 + \theta 5 \cdot x5$$

Question 1. Écrivez cette hypothèse sous forme matricielle h_θ(X) = X·θ. Quelle doit être la forme (shape) de X pour m = 24 étudiants et 5 features (n'oubliez pas la colonne de biais) ?

Question 2. Pourquoi ajoute-t-on une colonne de 1 à X plutôt que de traiter θ0 séparément ?

---

4. Partie 2, Fonction de coût

$J(\theta) = (1 / 2m) \cdot \Sigma\ [h_\theta(x^{(i)}) - y^{(i)}]^2 \quad pour\ i = 1\ à\ m$

Question 3. Pourquoi utiliser le carré de l'erreur plutôt que la valeur absolue ?

Question 4. À quoi sert le facteur 1/2 dans la formule ?

Question 5. Que représente géométriquement J(θ) dans l'espace des paramètres (pour un cas à 1 seule feature, esquissez la courbe) ?

---

5. Partie 3, Descente de gradient

À faire à la main (sur papier) avant de coder :

Question 6. Calculez ∂J/∂θj et montrez qu'on obtient :

$\partial J/\partial \theta j = (1/m) \cdot \Sigma\ [h_\theta(x^{(i)}) - y^{(i)}] \cdot xj^{(i)}$

Question 7. Écrivez la règle de mise à jour simultanée de tous les θj avec le taux d'apprentissage α :

$\theta j := \theta j - \alpha \cdot \partial J / \partial \theta j$

Question 8. Pourquoi la mise à jour doit-elle être simultanée (tous les θj calculés avec l'ancien vecteur θ, puis appliqués ensemble) plutôt que séquentielle ?

---

6. Partie 4, Implémentation (numpy, sans sklearn)

7. Partie 5, Expérimentation

Question 9. Tracez history (le coût J à chaque itération). La courbe doit être strictement décroissante et convexe, sinon, quel est le problème le plus probable ?

Question 10. Testez $\alpha \in \{0.001, 0.01, 0.05, 0.3, 1\}$. Que se passe-t-il pour les valeurs trop grandes ? Trop petites ?

Question 11. Relancez l'entraînement sans normaliser les features. Comparez la vitesse de convergence. Expliquez pourquoi la normalisation change autant le comportement de la descente de gradient (indice : forme des courbes de niveau de J).

Question 12. Comparez vos θ finaux à la solution analytique (équation normale) $\theta = (X^T X)^{-1} X^T y$. Sont-ils proches ? Sinon, augmentez n_iter ou ajustez alpha.
